Lab Practice Problem 11A

Filename: `No File Submission`

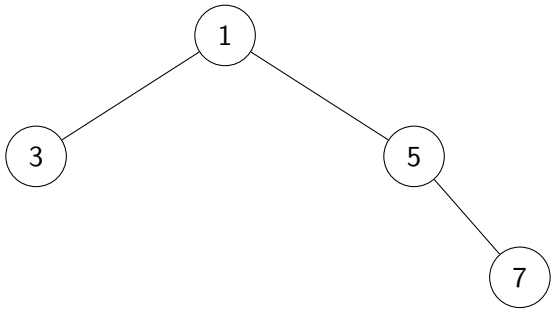
In this activity, you will work with binary heaps by determining whether given trees or arrays satisfy the min-heap property, constructing heap structures, and implementing heap-related operations in code. The goal of this exercise is to strengthen your understanding of heap properties.

Important: Do not run any program or use code to compute the results for this activity. The purpose of this exercise is to practice tracing the algorithm manually.

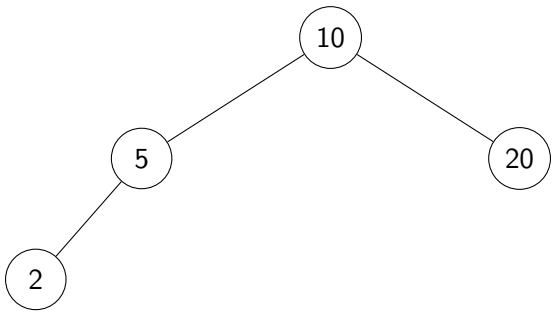
Binary Heaps (Min-Heaps)

Determine whether each given tree satisfies the properties of a min-heap. For each tree, answer Yes or No, and provide a brief justification.

Tree 1



Tree 2



Tree 3

$$4,\ 6,\ 3,\ 8,\ 1$$

Build Min-Heap Operation

Construct a min-heap from the given array.

$$250,\ 45,\ 200,\ 40,\ 150,\ 42,\ 130,\ 50,\ 30,\ 100,\ 10$$

Coding Exercise

Complete a non-recursive function that takes an array, along with its size and capacity, as input. The function should return 1 if the array represents a min-heap and 0 otherwise. Assume the root of the heap is stored at index 0.

Finally, analyze and state the function's running time using Big-O notation.

Hint: When a heap is stored in an array, the indices of the children and the parent depend on the indexing convention.

If the root is at index 0 (0-based indexing):

- Left child: $2i + 1$
- Right child: $2i + 2$
- Parent: $\left\lfloor \frac{i-1}{2} \right\rfloor$

If the root is at index 1 (1-based indexing):

- Left child: $2i$
- Right child: $2i + 1$
- Parent: $\left\lfloor \frac{i}{2} \right\rfloor$

Make sure to apply the correct formulas based on the indexing used.

```
int is_min_heap(int *arr, int size, int capacity) {



}
```

Algorithms

The following pseudocode demonstrates how to construct a min-heap and restore the min-heap property when violations occur.

Constructing a Min-Heap

This operation transforms an existing array into a valid min-heap.

```
BuildHeap(A, size):
    Let idx_last = size - 1
    Compute idx_parent of idx_last (the parent)
    Let idx = idx_parent

    While idx >= 0:
        Heapify(A, size, idx)
        Decrement idx
```

Heapify-Down to Fix Min-Heap Violations

Recall that this operation is also known as percolate or percolate-down.

```
Heapify(A, size, idx):
    Let min = idx
    Compute left_idx and right_idx (child nodes of idx)
    If left_idx < size and A[left_idx] < A[min], then min = left_idx
    If right_idx < size and A[right_idx] < A[min], then min = right_idx

    If min != idx:
        swap values A[idx] and A[min]
        Heapify(A, size, min)
```